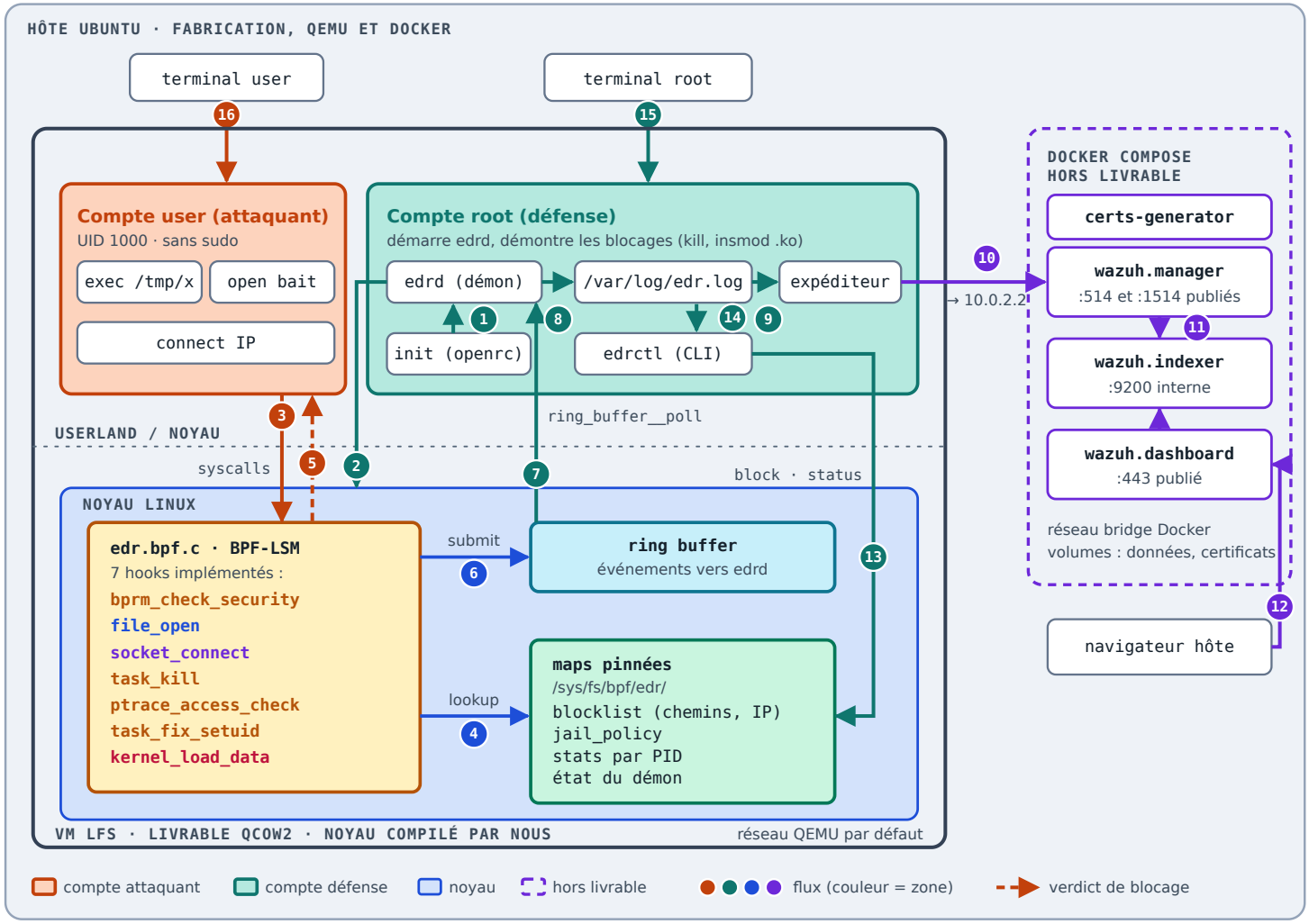
ANNEXE AU RAPPORT DE CONCEPTION · MINI EDR · NOYAU LINUX, ÉCOLE 2600

Architecture du Mini EDR

Détection et blocage in-kernel avec eBPF et BPF-LSM, alertes remontées vers un SIEM (Wazuh).

Vue d'ensemble



Détail des flux

N°	FLUX	MÉCANISME	CE QUI CIRCULE
1	init → edrd	Script openrc au démarrage : monte bpffs, puis lance `edrd`	Démarrage automatique de l'EDR, exigé par le sujet.
2	edrd → noyau	Appel système `bpf()` via libbpf : chargement, attache LSM, épinglage des maps et des liens BPF dans `/sys/fs/bpf/edr/`	Programmes BPF, maps et liens, chargés au boot en root.
3	user → hooks	Appels système : `execve`, `open`, `connect`	Les actions de l'attaquant. Chemin obligatoire : les tentatives de root en démo (`kill`, `insmod`) passent aussi par les hooks.
4	hooks ↔ maps	Lecture de la map par le programme BPF	PID, chemin ou IP comparés à la blocklist et à la `jail_policy`. Mise à jour des stats.
5	hooks → user	Valeur de retour du hook : 0 ou `-EPERM`	Le verdict. Synchrone : le syscall échoue avec `-EPERM` si l'action est bloquée.
6	hooks → ring buffer	`bpf_ringbuf_reserve` puis submit	Un événement : horodatage, PID, UID, `comm` et détail (chemin, adresse).
7	ring buffer → edrd	`ring_buffer__poll`	Les événements, en asynchrone. `edrd` est l'unique consumer.
8	edrd → edr.log	Écriture dans `/var/log/edr.log`	Une ligne par événement, en JSON (format proposé).
9	edr.log → expéditeur	Lecture en continu, par exemple `tail -F`	Les lignes du journal.
10	expéditeur → manager	TCP vers `10.0.2.2:514` (syslog)	Les lignes du journal. Sens unique, hors livrable.
11	manager, dashboard ↔ indexer	Réseau interne Docker	Le manager indexe les événements analysés. Le dashboard interroge l'indexer.
12	navigateur → dashboard	HTTPS sur le port 443 publié	La consultation des alertes par l'opérateur.
13	edrctl ↔ maps	Ouverture directe des maps épinglées, sans passer par `edrd`	`block add` et `block list` : règles. `status` : statistiques, état du démon (PID et horodatage de vie écrits par `edrd`) et liens épinglés, qui donnent la liste des hooks actifs.
14	edr.log → edrctl	Lecture du fichier en continu	Les alertes de `edrctl watch`. Choix de conception : le ring buffer n'a qu'un consumer, `edrd`.
15	terminal root → LFS	Console série QEMU, ou SSH via une redirection de port, à choisir	La session de root : lance `edrctl`, démontre les blocages.
16	terminal user → LFS	Même mécanisme, second terminal	La session de l'attaquant, ouverte en parallèle de celle de root.

Matrice des flux

source ↓ destination →	terminal	user	hooks BPF	maps pinnées	ring buffer	init	edrd	edr.log	expéditeur	edrctl	manager	indexer	dashboard	navigateur
terminal	—	16								15				
user		—	3	✕			✕	✕	✕		✕		✕	
hooks BPF		5	—	4	6									
maps pinnées			4	—						13				
ring buffer					—		7							
init						—	1							
edrd			2	2			—	8						
edr.log								—	9	14				
expéditeur									—		10			
edrctl				13						—				
manager											—	11		
indexer												—		
dashboard												11	—	
navigateur													12	—

n n n n flux autorisé (numéro du tableau précédent ; couleurs : attaquant, défense, noyau, SIEM) ✕ doit échouer

CELLULE	CE QUI DOIT L'EMPÊCHER, OU LA FAIBLESSE
user → maps pinnées	Répertoire `/sys/fs/bpf` en 0700 pour root, et appel `bpf()` réservé aux processus privilégiés : `sysctl kernel.unprivileged_bpf_disabled=1`.
user → edrd, expéditeur	Les permissions classiques d'abord. `task_kill` et `ptrace_access_check` couvrent un attaquant devenu root.
user → edr.log	Fichier en 0600, propriétaire root.
user → manager, dashboard	Risque : depuis `10.0.2.2`, le user pourrait envoyer de faux événements ou saturer le port syslog. Parade : le hook `socket_connect` refuse tout `connect` vers `10.0.2.2` dont l'UID n'est pas 0 (règle dans `jail_policy`). Limite : TCP uniquement, un envoi UDP passerait par `socket_sendmsg`, non couvert.

Choix de conception

Hooks implémentés

processus fichiers réseau modules

HOOK	APPEL INTERCEPTÉ	DÉCISION	MAP CONSULTÉE
`bprm_check_security`	execve	Refuse l'exécution depuis un chemin en blocklist, par exemple `/tmp`.	blocklist (chemins)
`file_open`	open	Refuse et journalise l'ouverture des fichiers leurres.	blocklist (chemins)
`socket_connect`	connect	Refuse les IP en blocklist et tout `connect` non root vers `10.0.2.2`.	blocklist (IP), jail_policy
`task_kill`	kill	Refuse l'envoi de signaux à `edrd` et à l'expéditeur.	jail_policy
`ptrace_access_check`	ptrace	Refuse le ptrace attach sur `edrd` et sur l'expéditeur.	jail_policy
`task_fix_setuid`	setuid	Journalise toute transition d'UID vers 0 (détection d'élévation de privilèges).	stats par PID
`kernel_load_data`	chargement de module	Refuse tout chargement de module, root compris, une fois la VM démarrée.	jail_policy

Prérequis noyau et système

ÉLÉMENT	RAISON
`CONFIG_BPF, CONFIG_BPF_SYSCALL`	Appel système `bpf()` (imposé par le sujet). `CONFIG_BPF_JIT` est ajouté pour la compilation des programmes.
`CONFIG_BPF_LSM=y`	Rend disponibles les hooks de sécurité BPF.
`CONFIG_LSM="...,bpf"`	`bpf` ajouté à la liste des LSM actifs (imposé par le sujet). Sans lui, les programmes BPF-LSM ne sont pas appelés. Le paramètre de démarrage `lsm=...,bpf` est l'équivalent.
`CONFIG_DEBUG_INFO_BTF=y`	Nécessaire à CO-RE (imposé par le sujet) et à BPF-LSM. Exige `pahole` à la compilation du noyau ; fournit `/sys/kernel/btf/vmlinux`.
`bpffs sur /sys/fs/bpf`	Pinning des maps. Monté par le script openrc avant `edrd` (flux 1).
`Hôte de fabrication`	clang/LLVM, `bpftool` et libbpf pour compiler `edr.bpf.c`.

Modes de défaillance

SITUATION	COMPORTEMENT	PARADE
`edrd plante ou est tué`	Les hooks restent attachés (liens BPF pinnés) et les maps pinnées survivent : le blocage continue. Les événements non consommés du ring buffer sont perdus.	openrc relance `edrd`, qui réutilise les maps pinnées. `task_kill` et `ptrace_access_check` le protègent.
`Ring buffer plein`	`bpf_ringbuf_reserve` échoue : l'événement est perdu, le verdict n'est pas affecté.	Ring buffer de 256 Kio. Compteur d'événements perdus dans la map de stats, affiché par `edrctl status`.
`Règle absente ou erreur de map`	Politique par défaut : fail-open, on autorise ce qui n'est pas listé. Un fail-closed rendrait la VM inutilisable au moindre bug.	Compteur d'erreurs dans la map de stats.
`edr.log grossit`	Risque de remplir le disque de la VM.	Rotation par taille (logrotate, 10 Mio, 5 archives). `edrd` rouvre le fichier sur SIGHUP ; `tail -F` suit le renommage.
`Manager Wazuh injoignable`	Aucune perte locale : `edr.log` reste la source de vérité.	L'expéditeur se reconnecte en boucle et reprend à la dernière position lue.